28. Algorithmic Problem Set (Mixed Drills)

28. Algorithmic Problem Set (Mixed Drills)

Phase: Phase 4 — Algorithms and data structures
Estimated time: 2–3 days
Slug: `28-algorithm-drills`

Learning objectives

- Solve mixed problems under constraints
- State complexity before coding
- Write tests first for edge cases
- Refactor brute force to efficient solutions

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Mixed drills build fluency: hashing + two pointers + sorting under time pressure. This is how interview and on-call problem solving get faster.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Clarify inputs/outputs and constraints before coding.
- Start with brute force, then improve with maps/sorts.
- Two-sum pattern: map value→index.
- Write tests first for edge cases: empty, duplicates, negatives.
- Idiomatic Go: early returns, no clever one-liners.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-algorithmic && cd /tmp/gw-algorithmic
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import "fmt"

func TwoSum(nums []int, target int) [2]int {
    idx := map[int]int{}
    for i, n := range nums {
        if j, ok := idx[target-n]; ok {
            return [2]int{j, i}
        }
        idx[n] = i
    }
    return [2]int{-1, -1}
}

func main() { fmt.Println(TwoSum([]int{2, 7, 11, 15}, 9)) }
```

Run it:

```
go run .
```

Expected output:

```
[0 1]
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Implement two-sum with a map.
2. Add tests for not-found and duplicates.
3. Time a naive O(n^2) vs map solution.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Drill protocol

For each problem: 1. Restate inputs/outputs and constraints 2. Work an example by hand 3. Propose brute force + complexity 4. Improve if needed 5. Implement in Go with table tests 6. Benchmark if n is large

Problem themes in the exercises

- Two-sum variants with maps
- Sliding window
- Interval merge
- Top-K with heaps
- Graph connectivity

Treat this lesson as a gym: correctness under pressure, then clarity.

Failure modes to watch while coding

- Coding before examples
- Silent assumption about sorted input
- Mutating inputs unexpectedly

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Same patterns; dict vs map syntax.
- enumerate ≈ for i, v := range.
- Return tuples vs arrays/slices — match the API.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Narrate complexity out loud while coding — it catches bad approaches early.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Algorithmic Problem Set (Mixed Drills)” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Coding before examples
- Silent assumption about sorted input
- Mutating inputs unexpectedly

Exercises

Work in `exercises/28-algorithm-drills/`. Read that folder's `README.md` before you code.

What you will build

Implement TwoSum (or the drill set) exactly as tests specify indices.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/28-algorithm-drills/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. What is your first step before code?
2. How do you pick a data structure?
3. When is brute force OK?

Further reading

- https://go.dev/blog/slices